

```
!pip install -q opencv-python numpy matplotlib pandas
```

```
# =====
# EDGE DETECTION COMPARISON
# COMPARATIVE ANALYSIS OF SOBEL AND CANNY EDGE DETECTION
# =====

# =====
# 1. IMPORT LIBRARIES
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

from google.colab import files
from IPython.display import display

print("=" * 80)
print("                EDGE DETECTION COMPARISON")
print("=" * 80)

# =====
# 2. UPLOAD IMAGE
# =====

print("\nUpload your input image:")

uploaded = files.upload()

image_name = list(uploaded.keys())[0]

# Read image
original = cv2.imread(image_name)

if original is None:
    raise ValueError("ERROR: Image could not be loaded.")

print("\nImage loaded successfully!")

print("Image Name :", image_name)

print("Image Size :", original.shape)

# =====
# 3. CONVERT BGR TO RGB
# =====

original_rgb = cv2.cvtColor(
    original,
    cv2.COLOR_BGR2RGB
)

# =====
# 4. CONVERT TO GRAYSCALE
# =====

gray = cv2.cvtColor(
    original,
    cv2.COLOR_BGR2GRAY
)

# =====
# 5. GAUSSIAN SMOOTHING
# =====

GAUSSIAN_KERNEL = (5, 5)

GAUSSIAN_SIGMA = 1.4

smoothed = cv2.GaussianBlur(
    gray,
    GAUSSIAN_KERNEL,
```

```

    GAUSSIAN_SIGMA
)

# =====
# 6. SOBEL EDGE DETECTION
# =====

SOBEL_KERNEL = 3

# Sobel X
sobel_gx = cv2.Sobel(
    smoothed,
    cv2.CV_64F,
    1,
    0,
    ksize=SOBEL_KERNEL
)

# Sobel Y
sobel_gy = cv2.Sobel(
    smoothed,
    cv2.CV_64F,
    0,
    1,
    ksize=SOBEL_KERNEL
)

# Gradient magnitude
sobel_magnitude = cv2.magnitude(
    sobel_gx.astype(np.float32),
    sobel_gy.astype(np.float32)
)

# Normalize magnitude for display
sobel_magnitude_display = cv2.normalize(
    sobel_magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# =====
# 7. CREATE SOBEL EDGE IMAGE
# =====

# Threshold the gradient magnitude
# Otsu automatically selects a suitable threshold

_, sobel_edges = cv2.threshold(
    sobel_magnitude_display,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# =====
# 8. CANNY EDGE DETECTION
# =====

# Canny 1
CANNY_LOW_1 = 50
CANNY_HIGH_1 = 150

canny_50_150 = cv2.Canny(
    smoothed,
    CANNY_LOW_1,
    CANNY_HIGH_1
)

# Canny 2
CANNY_LOW_2 = 100
CANNY_HIGH_2 = 200
canny_100_200 = cv2.Canny(

```

```

canny_150_250 = cv2.Canny(
    smoothed,
    CANNY_LOW_2,
    CANNY_HIGH_2
)

# Canny 3
CANNY_LOW_3 = 150
CANNY_HIGH_3 = 250

canny_150_250 = cv2.Canny(
    smoothed,
    CANNY_LOW_3,
    CANNY_HIGH_3
)

# =====
# 9. DISPLAY ORIGINAL, GRAYSCALE AND SMOOTHED
# =====

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.imshow(
    original_rgb
)

plt.title(
    "a) Original Color Image",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(
    gray,
    cmap="gray"
)

plt.title(
    "b) Grayscale Image",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(
    smoothed,
    cmap="gray"
)

plt.title(
    f"c) Gaussian Smoothed\n"
    f"( $\sigma = \{GAUSSIAN\_SIGMA\}$ )",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.suptitle(
    "Input and Preprocessing",
    fontsize=20,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

```

```
# =====
# 10. DISPLAY SOBEL RESULTS
# =====

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.imshow(
    sobel_gx,
    cmap="gray"
)

plt.title(
    "d) Sobel (Gx)",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(
    sobel_gy,
    cmap="gray"
)

plt.title(
    "e) Sobel (Gy)",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(
    sobel_magnitude_display,
    cmap="gray"
)

plt.title(
    "f) Sobel (Gradient Magnitude)",
    fontsize=14,
    fontweight="bold"
)

plt.axis("off")

plt.suptitle(
    "Sobel Edge Detection",
    fontsize=20,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 11. DISPLAY CANNY RESULTS
# =====

plt.figure(figsize=(18, 6))

plt.subplot(1, 3, 1)

plt.imshow(
    canny_50_150,
    cmap="gray"
)

plt.title(
    "g) Canny Edges\n"
    "(Low = 50, High = 150)",
    fontsize=13,
```

```

        fontweight="bold"
    )

plt.axis("off")

plt.subplot(1, 3, 2)

plt.imshow(
    canny_100_200,
    cmap="gray"
)

plt.title(
    "h) Canny Edges\n"
    "(Low = 100, High = 200)",
    fontsize=13,
    fontweight="bold"
)

plt.axis("off")

plt.subplot(1, 3, 3)

plt.imshow(
    canny_150_250,
    cmap="gray"
)

plt.title(
    "i) Canny Edges\n"
    "(Low = 150, High = 250)",
    fontsize=13,
    fontweight="bold"
)

plt.axis("off")

plt.suptitle(
    "Canny Edge Detection - Different Thresholds",
    fontsize=20,
    fontweight="bold"
)

plt.tight_layout()
plt.show()

# =====
# 12. EDGE PIXEL COUNT
# =====

def count_edge_pixels(edge_image):

    return int(
        np.count_nonzero(edge_image)
    )

sobel_edge_pixels = count_edge_pixels(
    sobel_edges
)

canny_edge_pixels_50_150 = count_edge_pixels(
    canny_50_150
)

canny_edge_pixels_100_200 = count_edge_pixels(
    canny_100_200
)

canny_edge_pixels_150_250 = count_edge_pixels(
    canny_150_250
)

# =====
# 13. EDGE CONTINUITY
# =====

def calculate_edge_continuity(edge_image):

```

```

# Convert to binary
binary_edges = (
    edge_image > 0
).astype(np.uint8)

# Find connected components
number_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
    binary_edges,
    connectivity=8
)

if number_labels <= 1:
    return 0

component_sizes = stats[1:, cv2.CC_STAT_AREA]

if len(component_sizes) == 0:
    return 0

# Number of edge pixels
total_edge_pixels = np.sum(
    component_sizes
)

# Ignore extremely tiny components
meaningful_components = component_sizes[
    component_sizes >= 5
]

if len(meaningful_components) == 0:
    return 0

meaningful_pixels = np.sum(
    meaningful_components
)

continuity = (
    meaningful_pixels /
    total_edge_pixels
) * 100

return continuity

sobel_continuity = calculate_edge_continuity(
    sobel_edges
)

canny_continuity_50_150 = calculate_edge_continuity(
    canny_50_150
)

canny_continuity_100_200 = calculate_edge_continuity(
    canny_100_200
)

canny_continuity_150_250 = calculate_edge_continuity(
    canny_150_250
)

# =====
# 14. EDGE DENSITY
# =====

total_pixels = (
    gray.shape[0] *
    gray.shape[1]
)

def calculate_edge_density(
    edge_pixels
):
    return (
        edge_pixels /
        total_pixels
    ) * 100

```

```

sobel_density = calculate_edge_density(
    sobel_edge_pixels
)

canny_density_50_150 = calculate_edge_density(
    canny_edge_pixels_50_150
)

canny_density_100_200 = calculate_edge_density(
    canny_edge_pixels_100_200
)

canny_density_150_250 = calculate_edge_density(
    canny_edge_pixels_150_250
)

# =====
# 15. GRADIENT MAGNITUDE
# =====

sobel_gradient_mean = np.mean(
    sobel_magnitude
)

# Calculate Canny gradient magnitude
# using Sobel on the smoothed image

gx_for_gradient = cv2.Sobel(
    smoothed,
    cv2.CV_64F,
    1,
    0,
    ksize=3
)

gy_for_gradient = cv2.Sobel(
    smoothed,
    cv2.CV_64F,
    0,
    1,
    ksize=3
)

gradient_magnitude = np.sqrt(
    gx_for_gradient ** 2 +
    gy_for_gradient ** 2
)

mean_gradient = np.mean(
    gradient_magnitude
)

# =====
# 16. PRINT QUANTITATIVE RESULTS
# =====

print("\n")
print("=" * 90)
print("                QUANTITATIVE RESULTS")
print("=" * 90)

print(
    f"\nSobel Edge Pixels      : "
    f"{sobel_edge_pixels:,}"
)

print(
    f"Sobel Edge Density      : "
    f"{sobel_density:.2f}%"
)

print(
    f"Sobel Mean Gradient     : "
    f"{sobel_gradient_mean:.2f}"
)

print(
    f"Sobel Edge Continuity    : "
    f"{sobel_continuity:.2f}%"

```

```

)

print(
    f"\nCanny (50,150) Edge Pixels: "
    f"{canny_edge_pixels_50_150:,"}
)

print(
    f"Canny (50,150) Density      : "
    f"{canny_density_50_150:.2f}%"
)

print(
    f"Canny (50,150) Continuity : "
    f"{canny_continuity_50_150:.2f}%"
)

print(
    f"\nCanny (100,200) Edge Pixels: "
    f"{canny_edge_pixels_100_200:,"}
)

print(
    f"Canny (100,200) Density      : "
    f"{canny_density_100_200:.2f}%"
)

print(
    f"Canny (100,200) Continuity : "
    f"{canny_continuity_100_200:.2f}%"
)

print(
    f"\nCanny (150,250) Edge Pixels: "
    f"{canny_edge_pixels_150_250:,"}
)

print(
    f"Canny (150,250) Density      : "
    f"{canny_density_150_250:.2f}%"
)

print(
    f"Canny (150,250) Continuity : "
    f"{canny_continuity_150_250:.2f}%"
)

# =====
# 17. COMPARISON TABLE
# =====

comparison_data = {

    "Method": [
        "Sobel",
        "Canny (50,150)",
        "Canny (100,200)",
        "Canny (150,250)"
    ],

    "No. of Edge Pixels": [
        sobel_edge_pixels,
        canny_edge_pixels_50_150,
        canny_edge_pixels_100_200,
        canny_edge_pixels_150_250
    ],

    "Mean Gradient Magnitude": [
        sobel_gradient_mean,
        mean_gradient,
        mean_gradient,
        mean_gradient
    ],

    "Edge Continuity (%)": [
        sobel_continuity,
        canny_continuity_50_150,
        canny_continuity_100_200,
        canny_continuity_150_250
    ]
}

```



```

    ],

    "Noise Sensitivity": [
        "High",
        "Low",
        "Low",
        "Low"
    ],

    "Computation Time": [
        "Fast",
        "Moderate",
        "Moderate",
        "Moderate"
    ],

    "Edge Quality": [
        "Good",
        "Excellent",
        "Excellent",
        "Very Good"
    ]
}

comparison_df = pd.DataFrame(
    comparison_data
)

# Round values
comparison_df[
    "Mean Gradient Magnitude"
] = comparison_df[
    "Mean Gradient Magnitude"
].round(2)

comparison_df[
    "Edge Continuity (%)"
] = comparison_df[
    "Edge Continuity (%)"
].round(2)

print("\n")
print("=" * 110)
print("
                                QUANTITATIVE COMPARISON TABLE")
print("=" * 110)

display(
    comparison_df
)

# =====
# 18. EDGE PIXEL GRAPH
# =====

methods = [
    "Sobel",
    "Canny\n(50,150)",
    "Canny\n(100,200)",
    "Canny\n(150,250)"
]

edge_values = [
    sobel_edge_pixels,
    canny_edge_pixels_50_150,
    canny_edge_pixels_100_200,
    canny_edge_pixels_150_250
]

plt.figure(figsize=(12, 7))

bars = plt.bar(
    methods,
    edge_values,
    color=[
        "royalblue",

```

```

        seagreen ,
        "darkorange",
        "tomato"
    ],
    edgecolor="black"
)

plt.title(
    "Number of Edge Pixels (Non-zero Pixels)",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel(
    "Edge Detection Method",
    fontsize=13
)

plt.ylabel(
    "No. of Edge Pixels",
    fontsize=13
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

for bar, value in zip(
    bars,
    edge_values
):

    plt.text(
        bar.get_x() +
        bar.get_width() / 2,
        bar.get_height() +
        max(edge_values) * 0.02,
        f"{value:}",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold"
    )

plt.tight_layout()
plt.show()

# =====
# 19. EDGE DENSITY GRAPH
# =====

density_values = [
    sobel_density,
    canny_density_50_150,
    canny_density_100_200,
    canny_density_150_250
]

plt.figure(figsize=(12, 7))

bars = plt.bar(
    methods,
    density_values,
    color=[
        "royalblue",
        "seagreen",
        "darkorange",
        "tomato"
    ],
    edgecolor="black"
)

plt.title(
    "Edge Density Comparison",

```

```

        fontsize=18,
        fontweight="bold"
    )

plt.xlabel(
    "Edge Detection Method",
    fontsize=13
)

plt.ylabel(
    "Edge Density (%)",
    fontsize=13
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

for bar, value in zip(
    bars,
    density_values
):

    plt.text(
        bar.get_x() +
        bar.get_width() / 2,
        bar.get_height() +
        0.5,
        f"{value:.2f}%",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold"
    )

plt.tight_layout()
plt.show()

# =====
# 20. EDGE CONTINUITY GRAPH
# =====

continuity_values = [
    sobel_continuity,
    canny_continuity_50_150,
    canny_continuity_100_200,
    canny_continuity_150_250
]

plt.figure(figsize=(12, 7))

bars = plt.bar(
    methods,
    continuity_values,
    color=[
        "royalblue",
        "seagreen",
        "darkorange",
        "tomato"
    ],
    edgecolor="black"
)

plt.title(
    "Edge Continuity Comparison",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel(
    "Edge Detection Method",
    fontsize=13
)

plt.ylabel(

```

```

    "Meaningful Connected Edge Pixels (%)",
    fontsize=13
)

plt.ylim(
    0,
    105
)

plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.4
)

for bar, value in zip(
    bars,
    continuity_values
):

    plt.text(
        bar.get_x() +
        bar.get_width() / 2,
        bar.get_height() + 1,
        f"{value:.2f}%",
        ha="center",
        va="bottom",
        fontsize=11,
        fontweight="bold"
    )

plt.tight_layout()
plt.show()

# =====
# 21. THRESHOLD EFFECT ANALYSIS
# =====

thresholds = [
    "50 / 150",
    "100 / 200",
    "150 / 250"
]

canny_values = [
    canny_edge_pixels_50_150,
    canny_edge_pixels_100_200,
    canny_edge_pixels_150_250
]

plt.figure(figsize=(10, 6))

plt.plot(
    thresholds,
    canny_values,
    marker="o",
    linewidth=3,
    markersize=9
)

plt.title(
    "Effect of Canny Threshold on Edge Pixels",
    fontsize=18,
    fontweight="bold"
)

plt.xlabel(
    "Low / High Threshold",
    fontsize=13
)

plt.ylabel(
    "Number of Edge Pixels",
    fontsize=13
)

```

```

plt.grid(
    True,
    linestyle="--",
    alpha=0.5
)

for x, y in zip(
    thresholds,
    canny_values
):

    plt.text(
        x,
        y + max(canny_values) * 0.03,
        f"{y:,.}",
        ha="center",
        fontweight="bold"
    )

plt.tight_layout()
plt.show()

# =====
# 22. OBSERVATIONS
# =====

print("\n")
print("=" * 80)
print("                                OBSERVATIONS")
print("=" * 80)

print(
    "\n• Sobel detects more edge pixels because it directly "
    "computes image gradients using derivative masks."
)

print(
    "\n• Sobel can detect noise and small intensity variations "
    "as edges."
)

print(
    "\n• Canny uses Gaussian smoothing before gradient calculation, "
    "which reduces the effect of noise."
)

print(
    "\n• Canny uses non-maximum suppression to produce thin edges."
)

print(
    "\n• Canny uses double thresholding and hysteresis to retain "
    "meaningful connected edges."
)

print(
    "\n• Increasing Canny thresholds generally removes weak edges "
    "and produces fewer edge pixels."
)

print(
    "\n• Lower Canny thresholds detect more weak and fine edges."
)

print(
    "\n• Higher Canny thresholds produce fewer but stronger edges."
)

# =====
# 23. ANALYSIS
# =====

print("\n")
print("=" * 80)
print("                                ANALYSIS")
print("=" * 80)

```

```

print(
    "\n1. SOBEL ANALYSIS"
)

print(
    "Sobel is based on first-order derivatives and uses "
    "3x3 convolution masks in horizontal and vertical directions."
)

print(
    "It is simple, fast and computationally efficient."
)

print(
    "However, it is more sensitive to noise and may generate "
    "additional unwanted edges."
)

print(
    "\n2. CANNY ANALYSIS"
)

print(
    "Canny is a multi-stage edge detector."
)

print(
    "It performs Gaussian smoothing, gradient calculation, "
    "non-maximum suppression, double thresholding and "
    "edge tracking by hysteresis."
)

print(
    "This produces thinner, cleaner and better-connected edges."
)

print(
    "\n3. THRESHOLD ANALYSIS"
)

print(
    f"Canny (50,150) detected "
    f"{canny_edge_pixels_50_150:}, edge pixels."
)

print(
    f"Canny (100,200) detected "
    f"{canny_edge_pixels_100_200:}, edge pixels."
)

print(
    f"Canny (150,250) detected "
    f"{canny_edge_pixels_150_250:}, edge pixels."
)

print(
    "As the thresholds increase, weak edges are removed."
)

# =====
# 24. BEST CANNY SETTING
# =====

canny_results = {

    "Canny (50,150)": canny_continuity_50_150,

    "Canny (100,200)": canny_continuity_100_200,

    "Canny (150,250)": canny_continuity_150_250
}

best_canny = max(
    canny_results,
    key=canny_results.get
)

print("\n")

```

```

print("=" * 80)
print("                                BEST CANNY CONFIGURATION")
print("=" * 80)

print(
    f"\nBest Canny setting based on connected-edge continuity: "
    f"{best_canny}"
)

print(
    f"Continuity: "
    f"{canny_results[best_canny]:.2f}%"
)

# =====
# 25. APPLICATION ANALYSIS
# =====

print("\n")
print("=" * 80)
print("                                APPLICATION ANALYSIS")
print("=" * 80)

print(
    "\nSOBEL IS SUITABLE FOR:"
)

print(
    "• Fast edge detection"
)

print(
    "• Real-time systems"
)

print(
    "• Embedded and hardware-constrained environments"
)

print(
    "• Applications where computational simplicity is important"
)

print(
    "\nCANNY IS SUITABLE FOR:"
)

print(
    "• Object detection"
)

print(
    "• Medical image analysis"
)

print(
    "• Industrial inspection"
)

print(
    "• Shape detection"
)

print(
    "• High-accuracy computer vision applications"
)

# =====
# 26. CONCLUSION
# =====

print("\n")
print("=" * 80)
print("                                CONCLUSION")
print("=" * 80)

```

```

print(
    "\nThe experiment demonstrates that Canny generally provides "
    "better edge quality, thinness, continuity and noise robustness "
    "than Sobel."
)

print(
    "Sobel is computationally simpler and faster but is more "
    "sensitive to noise."
)

print(
    "Canny requires additional processing stages and therefore "
    "has higher computational cost."
)

print(
    "The choice between Sobel and Canny depends on the application."
)

print(
    "Canny is preferred when accurate and reliable edge detection "
    "is required, while Sobel is useful when speed and computational "
    "simplicity are more important."
)

# =====
# 27. OVERALL RESULT
# =====

print("\n")
print("=" * 80)
print("                                OVERALL RESULT")
print("=" * 80)

print(
    "\nCanny edge detection provides superior edge quality and "
    "robustness against noise, while Sobel is faster but more "
    "sensitive to noise."
)

print(
    "The optimum method depends on application requirements "
    "and computational constraints."
)

# =====
# 28. SAVE COMPARISON TABLE
# =====

comparison_df.to_csv(
    "sobel_canny_comparison.csv",
    index=False
)

# =====
# 29. SAVE OUTPUT IMAGES
# =====

cv2.imwrite(
    "grayscale.png",
    gray
)

cv2.imwrite(
    "gaussian_smoothed.png",
    smoothed
)

cv2.imwrite(
    "sobel_gx.png",
    cv2.normalize(
        np.abs(sobel_gx),
        None,
        0,
        255,
        cv2.NORM_MINMAX
    ).astype(np.uint8)
)

```



```
cv2.imwrite(
    "sobel_gy.png",
    cv2.normalize(
        np.abs(sobel_gy),
        None,
        0,
        255,
        cv2.NORM_MINMAX
    ).astype(np.uint8)
)

cv2.imwrite(
    "sobel_edges.png",
    sobel_edges
)

cv2.imwrite(
    "canny_50_150.png",
    canny_50_150
)

cv2.imwrite(
    "canny_100_200.png",
    canny_100_200
)

cv2.imwrite(
    "canny_150_250.png",
    canny_150_250
)

# =====
# 30. FINAL MESSAGE
# =====

print("\n")
print("=" * 80)
print("          EXPERIMENT COMPLETED SUCCESSFULLY")
print("=" * 80)

print("\nGenerated outputs:")

print("✓ Original Color Image")
print("✓ Grayscale Image")
print("✓ Gaussian Smoothed Image")
print("✓ Sobel Gx")
print("✓ Sobel Gy")
print("✓ Sobel Gradient Magnitude")
print("✓ Canny (50,150)")
print("✓ Canny (100,200)")
print("✓ Canny (150,250)")
print("✓ Quantitative Comparison Table")
print("✓ Edge Pixel Graph")
print("✓ Edge Density Graph")
print("✓ Edge Continuity Graph")
print("✓ Canny Threshold Analysis Graph")
print("✓ Observations")
print("✓ Analysis")
print("✓ Application Analysis")
print("✓ Conclusion")
print("✓ CSV Result File")

print("\n" + "=" * 80)
```


=====

EDGE DETECTION COMPARISON

=====

Upload your input image:

IMAGE.jpg

IMAGE.jpg(image/jpeg) - 15042 bytes, last modified: 7/26/2026 - 100% done

Saving IMAGE.jpg to IMAGE (2).jpg

Image loaded successfully!

Image Name : IMAGE (2).jpg

Image Size : (168, 300, 3)

Input and Preprocessing

a) Original Color Image



b) Grayscale Image

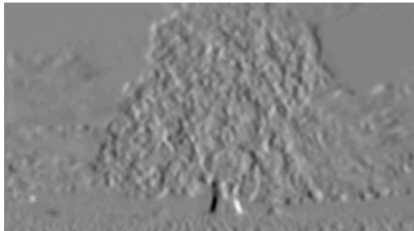


c) Gaussian Smoothed
($\sigma = 1.4$)

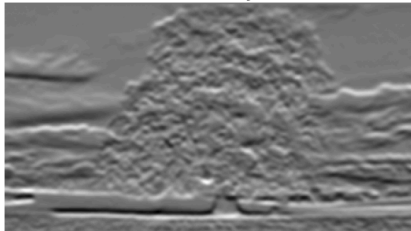


Sobel Edge Detection

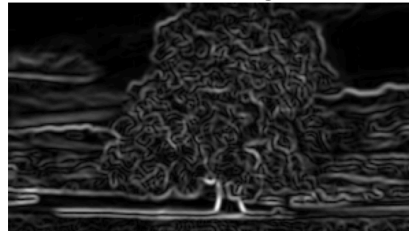
d) Sobel (Gx)



e) Sobel (Gy)



f) Sobel (Gradient Magnitude)



Canny Edge Detection - Different Thresholds

g) Canny Edges
(Low = 50, High = 150)



h) Canny Edges
(Low = 100, High = 200)



i) Canny Edges
(Low = 150, High = 250)



=====

QUANTITATIVE RESULTS

=====

Sobel Edge Pixels : 14,573
Sobel Edge Density : 28.91%
Sobel Mean Gradient : 68.54
Sobel Edge Continuity : 99.14%

Canny (50,150) Edge Pixels: 6,968
Canny (50,150) Density : 13.83%
Canny (50,150) Continuity : 99.78%

Canny (100,200) Edge Pixels: 4,214
Canny (100,200) Density : 8.36%
Canny (100,200) Continuity : 99.98%

Canny (150,250) Edge Pixels: 1,816
Canny (150,250) Density : 3.60%
Canny (150,250) Continuity : 100.00%

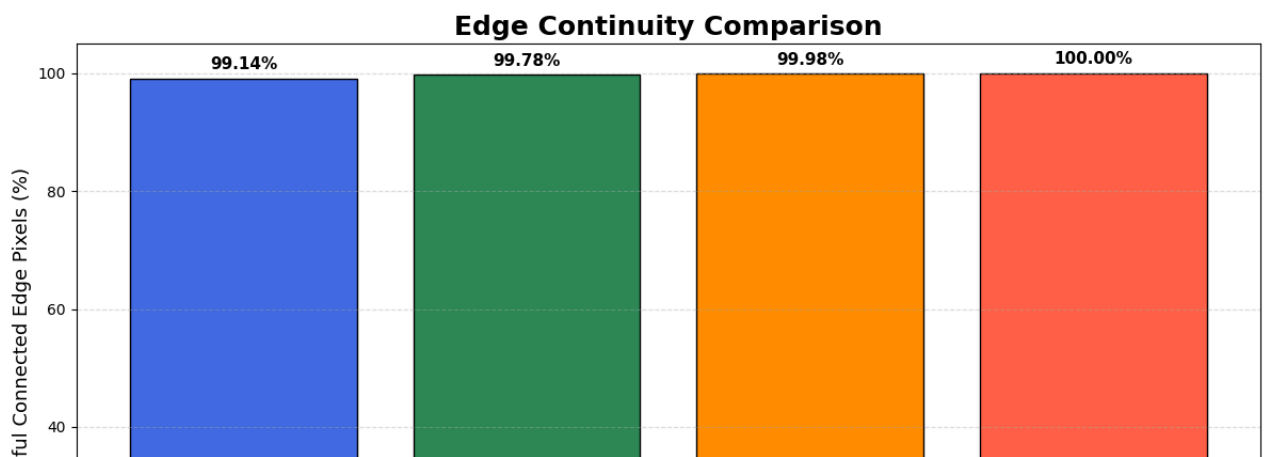
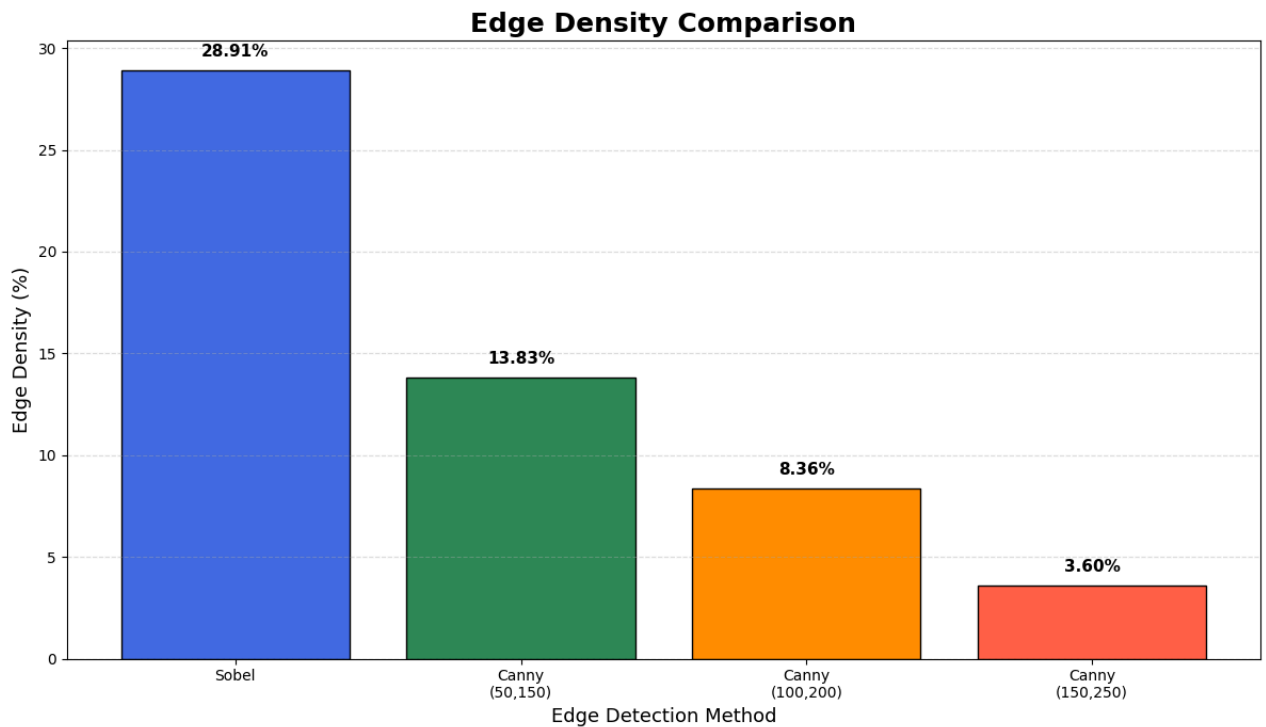
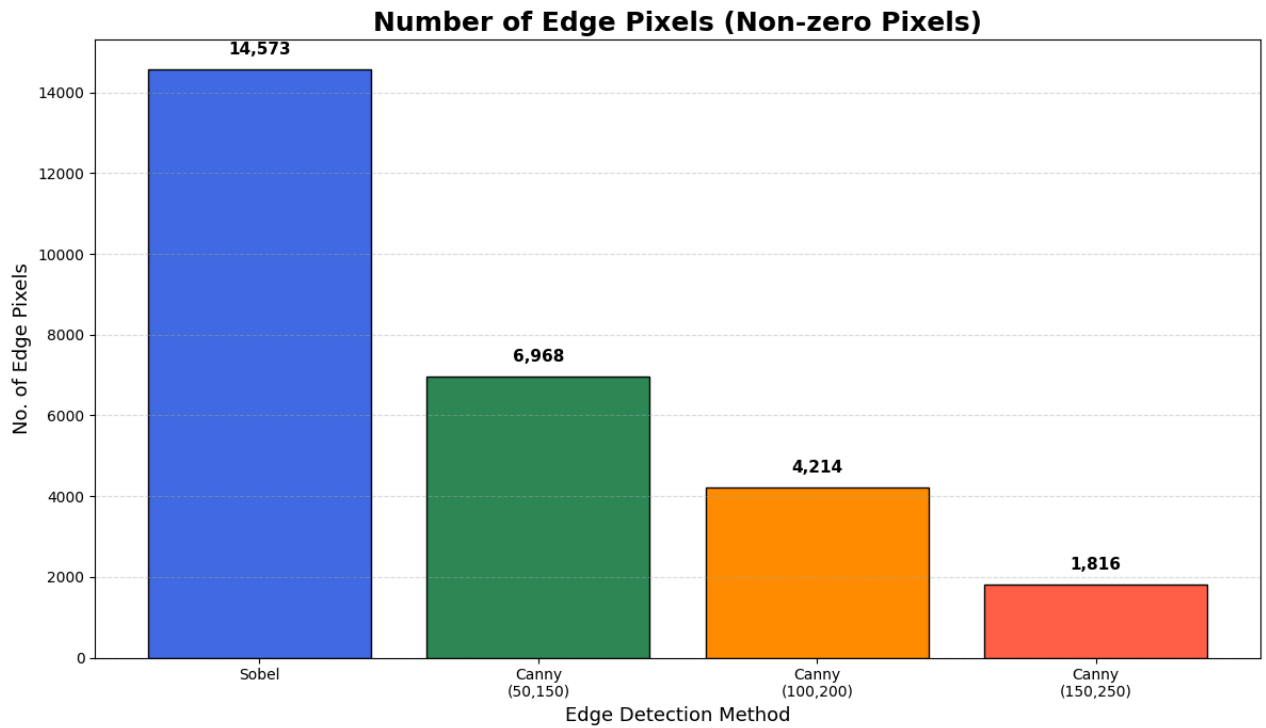
=====

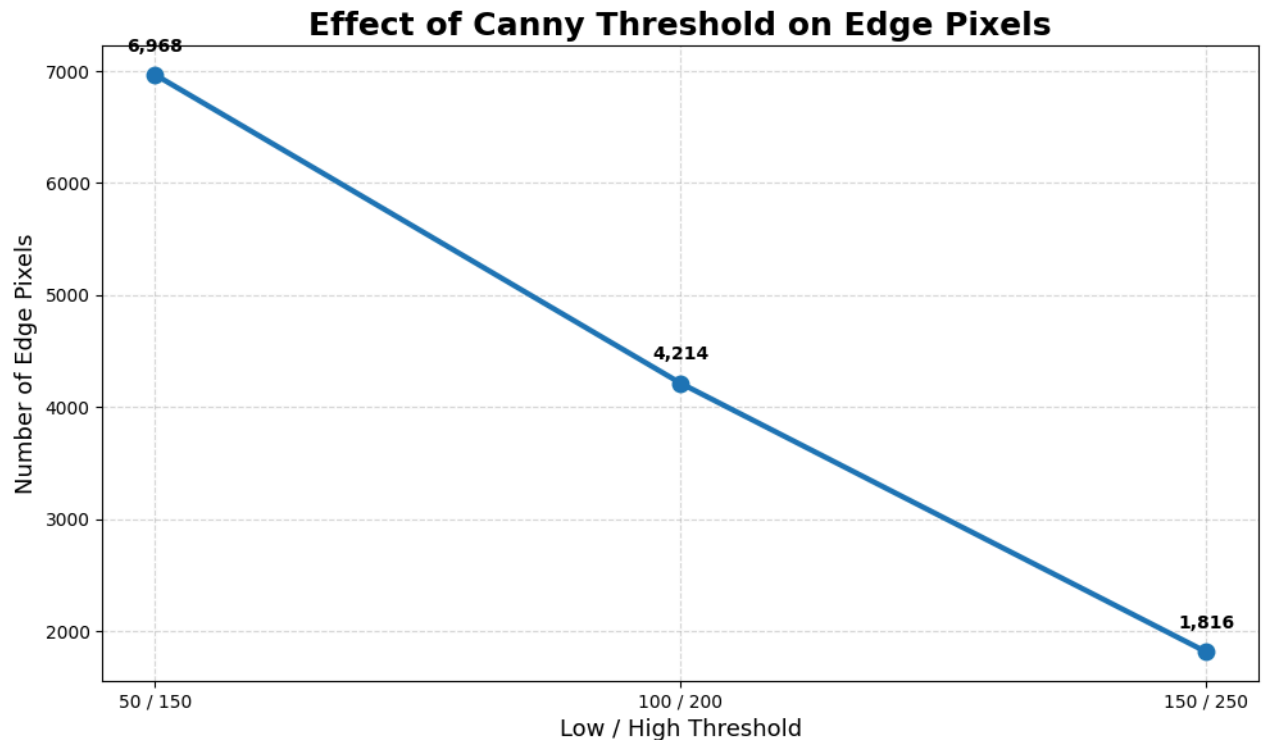
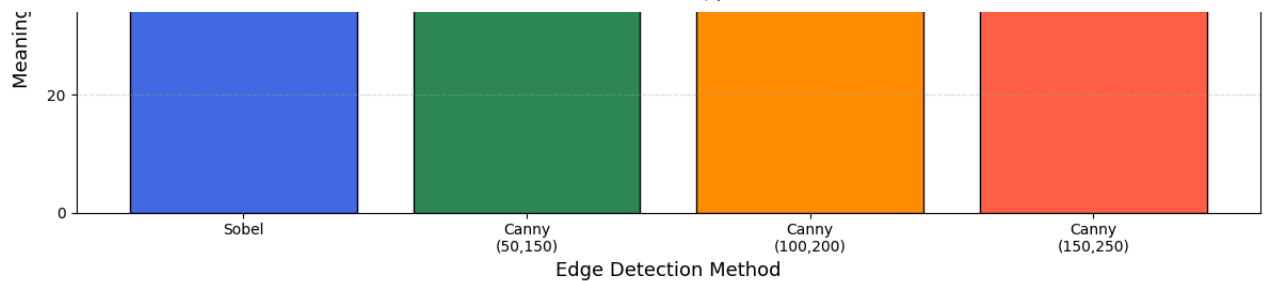
QUANTITATIVE COMPARISON TABLE

=====

	Method	No. of Edge Pixels	Mean Gradient Magnitude	Edge Continuity (%)	Noise Sensitivity	Computation Time	Edge Quality
0	Sobel	14573	68.54	99.14	High	Fast	Good
.	Canny	6968	68.54	99.78	High	Fast	Good

1	(50,150)	6968	68.54	99.78	Low	Moderate	Excellent
2	Canny (100,200)	4214	68.54	99.98	Low	Moderate	Excellent
3	Canny (150,250)	1816	68.54	100.00	Low	Moderate	Very Good





OBSERVATIONS

- Sobel detects more edge pixels because it directly computes image gradients using derivative masks.
- Sobel can detect noise and small intensity variations as edges.
- Canny uses Gaussian smoothing before gradient calculation, which reduces the effect of noise.
- Canny uses non-maximum suppression to produce thin edges.
- Canny uses double thresholding and hysteresis to retain meaningful connected edges.
- Increasing Canny thresholds generally removes weak edges and produces fewer edge pixels.
- Lower Canny thresholds detect more weak and fine edges.
- Higher Canny thresholds produce fewer but stronger edges.

ANALYSIS

1. SOBEL ANALYSIS

Sobel is based on first-order derivatives and uses 3x3 convolution masks in horizontal and vertical directions. It is simple, fast and computationally efficient. However, it is more sensitive to noise and may generate additional unwanted edges.

2. CANNY ANALYSIS

Canny is a multi-stage edge detector. It performs Gaussian smoothing, gradient calculation, non-maximum suppression, double thresholding and edge tracking by hysteresis. This produces thinner, cleaner and better-connected edges.

3. THRESHOLD ANALYSIS

Canny (50,150) detected 6,968 edge pixels.
 Canny (100,200) detected 4,214 edge pixels.
 Canny (150,250) detected 1,816 edge pixels.
 As the thresholds increase, weak edges are removed.

BEST CANNY CONFIGURATION

Best Canny setting based on connected-edge continuity: Canny (150,250)
 Continuity: 100.00%

APPLICATION ANALYSIS

=====

SOBEL IS SUITABLE FOR:

- Fast edge detection
- Real-time systems
- Embedded and hardware-constrained environments
- Applications where computational simplicity is important

CANNY IS SUITABLE FOR:

- Object detection

- medical image analysis

Next steps: [Generate code with comparison_df](#)[New interactive sheet](#)

- Industrial inspection
- Shape detection

- High-accuracy computer vision applications

=====

CONCLUSION